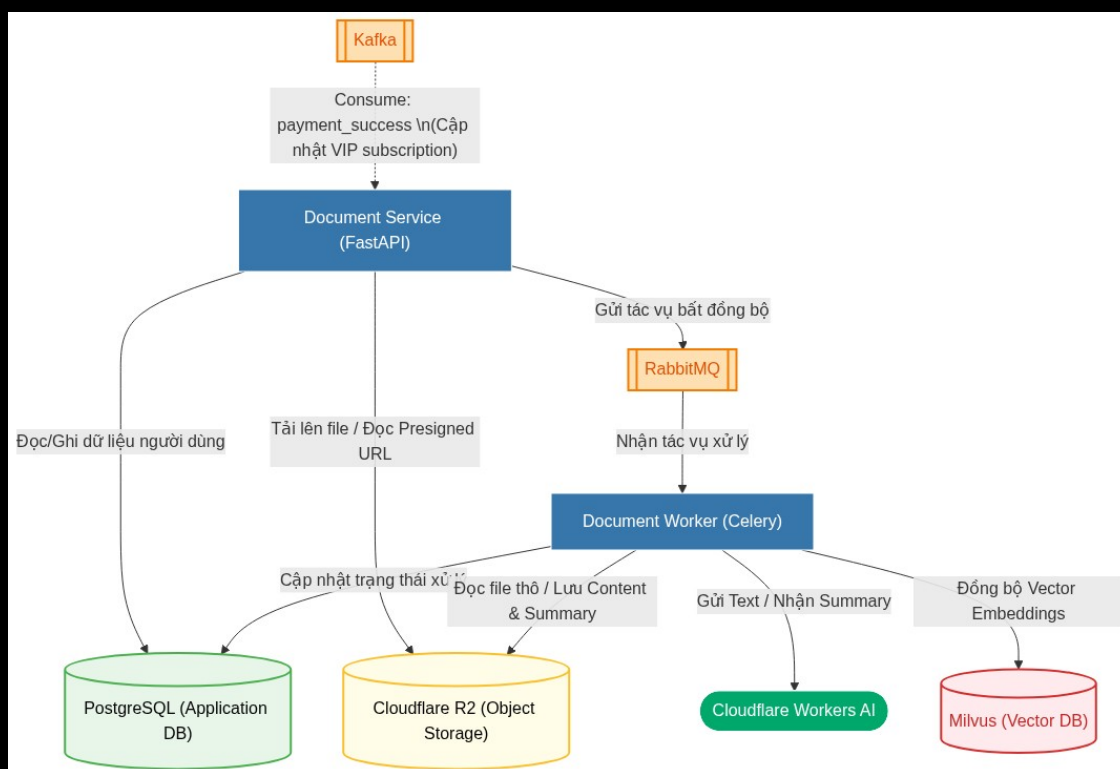


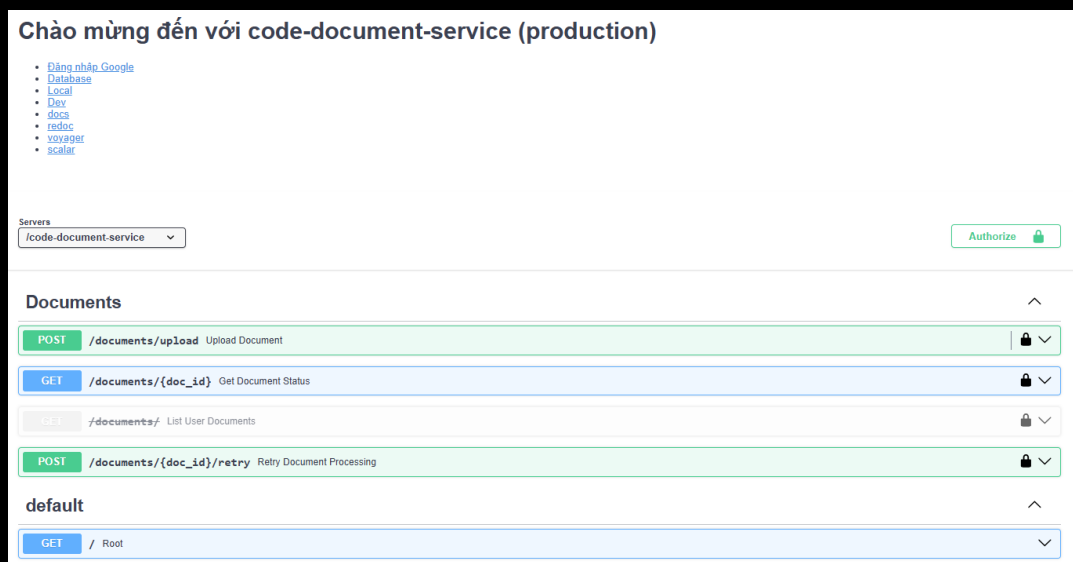
0.0.1 Dịch vụ văn bản (document service)

Sơ đồ tổng quan về Dịch vụ văn bản (Document Service):



Hình 0.1: Sơ đồ tổng quan Dịch vụ văn bản

Mục đích: Tiếp nhận, lưu trữ và quản lý luồng xử lý tài liệu của người dùng. Dịch vụ này không chỉ lưu trữ tệp tin mà còn hỗ trợ quy trình trích xuất nội dung sang định dạng Markdown, tự động tạo bản tóm tắt và thực hiện vector hóa tài liệu.



Hình 0.2: Tài liệu Swagger API Dịch vụ văn bản

Vì chức năng tải lên tài liệu chỉ dành riêng cho người dùng sở hữu gói VIP, dịch vụ văn bản (Document Service) thực hiện lắng nghe sự kiện thanh toán thành công từ Kafka, lưu thông tin vào CSDL để kiểm tra và xác thực quyền VIP của người dùng hiện tại.

- **Tải lên tài liệu mới:** Người dùng gửi tệp tài liệu lên hệ thống. Sau khi tiếp nhận thành công, hệ thống sẽ trả về mã định danh duy nhất (`doc_id`), tên tệp gốc và trạng thái khởi

tạo ban đầu, đồng thời đưa tác vụ vào hàng đợi xử lý.

- **Truy xuất trạng thái xử lý:** Dựa vào `doc_id`, người dùng có thể truy vấn trạng thái xử lý hiện tại của tài liệu. Hệ thống sẽ báo cáo chi tiết về sự tồn tại của các thành phần dữ liệu thông qua các cờ trạng thái: `has_file`, `has_content`, `has_summary`.
- **Thử lại tác vụ lỗi:** Trong trường hợp quá trình bóc tách hoặc tóm tắt gặp sự cố, hệ thống cung cấp một API chuyên biệt để kích hoạt lại quy trình xử lý cho tài liệu đó mà không cần người dùng phải tải lên lại tệp gốc.

Document Worker Dịch vụ văn bản kết hợp với **Document Worker** (nhận các tác vụ nền từ Celery thông qua RabbitMQ) để thực hiện quy trình xử lý tài liệu phục vụ hệ thống RAG:

1. **Tải tệp tin:** Tải tệp tin từ Cloudflare R2 và sử dụng thư viện `Docling` để bóc tách cấu trúc và nội dung văn bản.
2. **Tóm tắt văn bản:** Sử dụng mô hình ngôn ngữ trên Cloudflare Workers AI để tự động tạo bản tóm tắt nội dung tài liệu bằng tiếng Việt.
3. **Chia nhỏ văn bản (Chunking):** Sử dụng bộ chia `RecursiveCharacterTextSplitter` của `LangChain` để chia nhỏ văn bản thành các đoạn phù hợp.
4. **Trích xuất đặc trưng (Embedding):** Tạo vector biểu diễn ngữ nghĩa cho từng đoạn văn bản bằng mô hình `keepitreal/vietnamese-sbert`.
5. **Lưu trữ Vector:** Quản lý và lưu trữ các vector embedding vào CSDL vector Milvus để phục vụ tìm kiếm ngữ nghĩa sau này.

Cơ chế tối ưu & Độ bền bỉ (Optimization & Resilience)

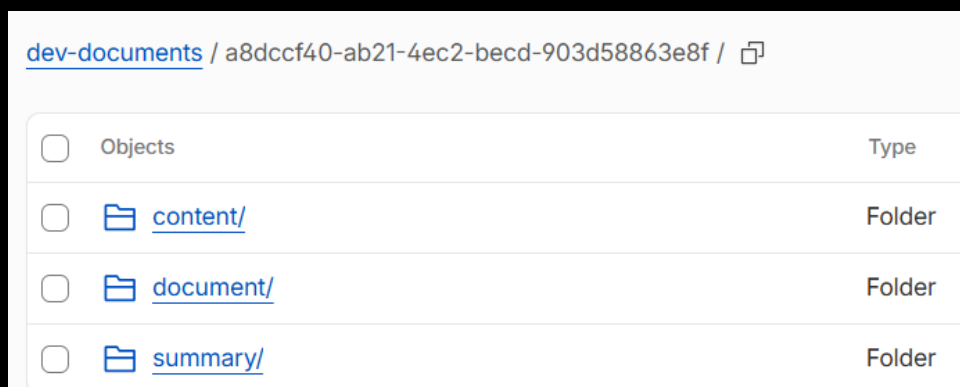
- **Tái sử dụng kết quả (Deduplication):** Để tối ưu chi phí và hiệu năng, trước khi chạy parser và AI, worker kiểm tra xem người dùng đã từng tải lên tệp tin có cùng dung lượng (`file_size`) và băm (`file_hash`) hay chưa. Nếu đã có tài liệu trùng khớp đã xử lý thành công, hệ thống sao chép trực tiếp kết quả bóc tách và tóm tắt trên Cloudflare R2 sang đường dẫn của tài liệu mới mà không cần chạy lại mô hình AI.
- **Cấu hình hàng đợi Celery bền bỉ:** Thiết lập `task_acks_late=True` để task chỉ được xóa khỏi RabbitMQ khi worker đã hoàn thành thực tế (tránh mất mát tác vụ); thiết lập `worker_prefetch_multiplier=1` ép worker chỉ nhận 1 tài liệu tại một thời điểm để cân bằng tải; tự động thử lại tối đa 3 lần (mỗi lần chờ 60 giây) nếu gặp sự cố kết nối ngoại vi.
- **Giám sát Langfuse:** Toàn bộ luồng tóm tắt văn bản bằng AI (Cloudflare Workers AI) được theo dõi vết và giám sát chi tiết thông qua nền tảng **Langfuse** để kiểm soát chất lượng và độ trễ.

Giao diện quản lý vector trên Milvus:



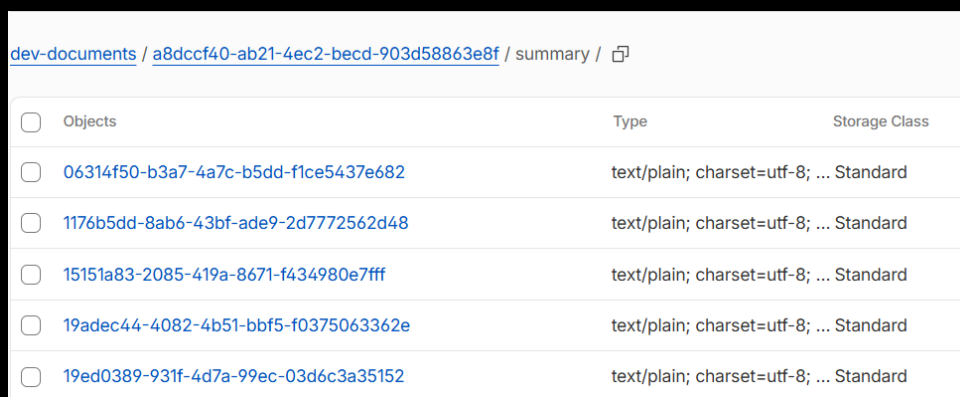
Hình 0.3: Dữ liệu vector lưu trữ trên Milvus

Quản lý các file tài liệu gốc trên Cloudflare R2:



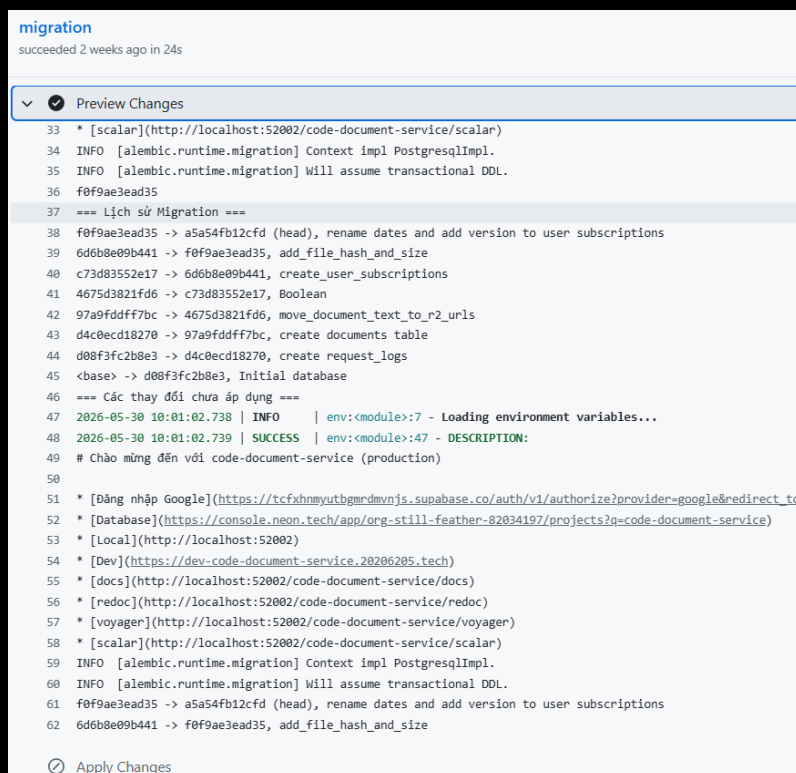
Hình 0.4: Tập tài liệu gốc lưu trữ trên Cloudflare R2

Quản lý nội dung tóm tắt lưu trữ trên Cloudflare R2:



Hình 0.5: Bản tóm tắt lưu trữ trên Cloudflare R2

Quản lý migration CSDL của dịch vụ văn bản:



Hình 0.6: Quản lý migration CSDL của dịch vụ văn bản

Kết quả các bảng trong CSDL tài liệu:

Tables	main
neondb	
Schema	
public	
alembic_version	version_num varchar(32) a5a54fb12cfd
documents	
request_logs	
user_subscriptions	

Hình 0.7: Kết quả các bảng trong CSDL tài liệu

0.0.2 Dịch vụ nhân vật (persona service)

Mục đích: Chịu trách nhiệm quản lý hệ thống nhân vật (personas), kho giọng đọc (Voices), các nền tảng tổng hợp giọng nói (TTS Engines) và xử lý trực tiếp các yêu cầu chuyển đổi văn bản thành âm thanh (Text-to-Speech).

Servers	code-persona-service	Authorize
Public		
GET	/personas	Get All Persona
Persona Admin		
POST	/personas	Create Persona
PUT	/personas/{persona_id}	Update Persona
DELETE	/personas/{persona_id}	Delete Persona
POST	/personas/upload-avatar	Upload Persona Avatar
POST	/personas/upload-audio	Upload Persona Audio
Audio		
GET	/audio/v1/	Root
POST	/audio/v1/audio/speech	Generate Speech
POST	/audio/v1/tts	Admin Generate Audio
Engine Admin		
Voice Admin		

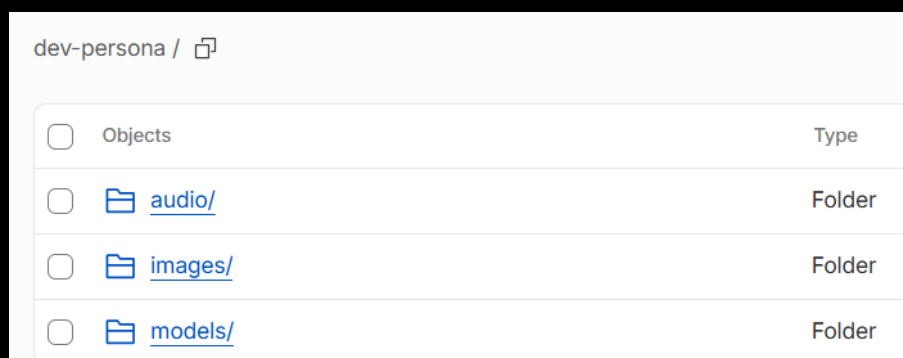
Hình 0.8: Tài liệu Swagger API Dịch vụ nhân vật

Các chức năng chính

- **Truy xuất danh sách nhân vật:** Cung cấp danh sách các nhân vật trong hệ thống, hỗ trợ phân trang và lọc theo `persona_id`.
- **Quản lý nhân vật (Dành cho ADMIN):** Cho phép tạo mới, cập nhật thông tin hoặc xóa bỏ các nhân vật khỏi hệ thống. Tải lên hình ảnh đại diện (**avatar**) và file âm thanh câu chào mẫu (welcome audio) cho từng nhân vật.
- **Quản lý công cụ TTS (engines):** Cho phép quản trị viên ADMIN định nghĩa, cập nhật, xóa và truy xuất danh sách các nền tảng dịch vụ cung cấp TTS.
- **Quản lý giọng đọc (voices):** Cho phép tạo, cập nhật, xóa và truy xuất danh sách các giọng đọc cụ thể, hỗ trợ bộ lọc theo công cụ/nền tảng.
- **Đồng bộ ElevenLabs:** Tích hợp API đồng bộ danh sách giọng đọc mới nhất từ nhà cung cấp ElevenLabs trực tiếp về hệ thống CSDL.

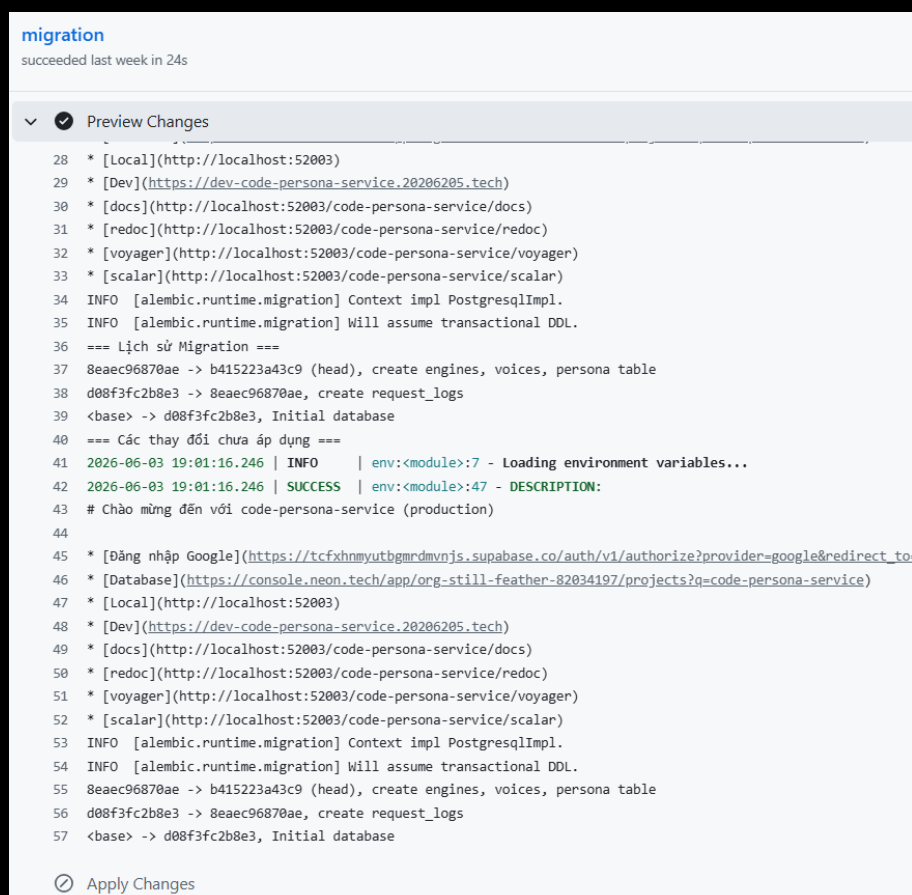
- **Chuyển đổi Text-to-Speech:** Cung cấp chức năng tạo ra âm thanh từ văn bản đầu vào thông qua hai thư viện: **edge-tts** (tổng hợp online) và **piper-tts** (tổng hợp offline chất lượng cao). Với **piper-tts**, hệ thống yêu cầu tệp mô hình giọng đọc được lưu trữ tại Cloudflare R2 và tự động tải xuống máy chủ cục bộ khi dịch vụ bắt đầu khởi động.
- **Cơ chế Cache tối ưu (TTS Caching):** Sử dụng **Disk Cache** (thư viện **diskcache** cơ chế LRU, kích thước giới hạn 1GB) để lưu giữ các file âm thanh đã tổng hợp lên ổ đĩa nhằm phần hồi tức thì các yêu cầu trùng lặp; và sử dụng **Memory Cache** để lưu giữ các đối tượng mô hình Piper đã load vào bộ nhớ RAM (**_loaded_voices**), tránh việc phải nạp lại tệp tin mô hình **.onnx** nặng sau mỗi yêu cầu, tối ưu tốc độ phát âm thanh.
- **Lưu trữ tài nguyên:** Toàn bộ dữ liệu mẫu âm thanh (**audio**), hình ảnh (**images**) và mô hình (**models**) phục vụ cho dịch vụ nhân vật được lưu trữ an toàn tại Cloudflare R2.

Quản lý thư mục tài nguyên của dịch vụ nhân vật trên Cloudflare R2:



Hình 0.9: Tài nguyên nhân vật lưu trữ trên Cloudflare R2

Quản lý migrations CSDL của dịch vụ nhân vật:



Hình 0.10: Quản lý migration CSDL của dịch vụ nhân vật

Cấu trúc các bảng trong CSDL của Dịch vụ nhân vật:

Tables

main

neondb

Schema

public

alembic_version

engines

persona

request_logs

voices

DATA

STRUCTURE

<

>

Filters

Sort

Columns

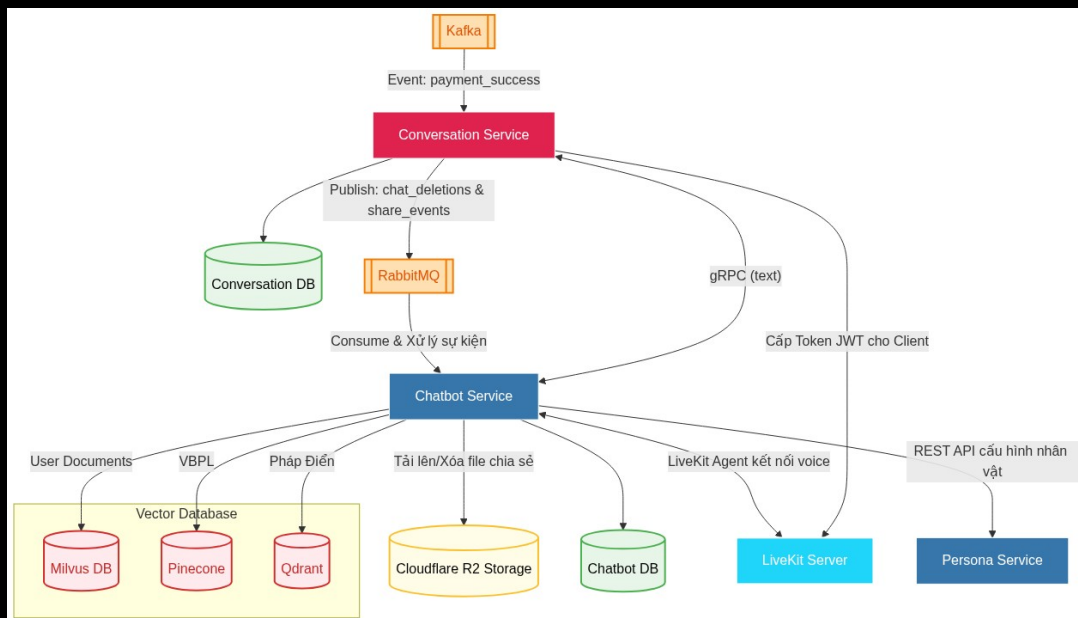
+ Add record

☐	id uuid	☐	code varchar(50)	☐	name varchar(100)	☐	is_active boolean	☐	voices
☐	a12e3456-789a-bcde-f01...	☐	edge_tts	☐	Edge TTS	☐	TRUE	☐	voices
☐	b12e3456-789a-bcde-f01...	☐	elevenlabs	☐	ElevenLabs	☐	TRUE	☐	voices
☐	e12e3456-789a-bcde-f01...	☐	pipertts	☐	Piper TTS	☐	TRUE	☐	voices

Hình 0.11: Cấu trúc các bảng trong CSDL của Dịch vụ nhân vật

0.0.3 Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service)

Sơ đồ tổng quan về Dịch vụ trò chuyện (conversation service) và Dịch vụ chatbot (chatbot service):



Hình 0.12: Sơ đồ tổng quan Dịch vụ trò chuyện và Dịch vụ chatbot

Dịch vụ trò chuyện (conversation service) Vì chức năng trò chuyện văn bản suy luận (reasoning) và trò chuyện bằng giọng nói chỉ dành riêng cho tài khoản VIP, nên dịch vụ trò chuyện (conversation service) thực hiện lắng nghe sự kiện thanh toán thành công từ Kafka, lưu lại thông tin vào CSDL và kiểm tra quyền VIP của người dùng hiện tại trước khi xử lý yêu cầu.

Mục đích: Chịu trách nhiệm quản lý luồng giao tiếp giữa người dùng và hệ thống Agentic RAG. Dịch vụ này xử lý trực tiếp các cuộc trò chuyện văn bản theo cơ chế streaming, cấp phát token truy cập cho luồng giao tiếp giọng nói và cung cấp các tiện ích lưu trữ, chia sẻ trò chuyện.

Các tính năng chính:

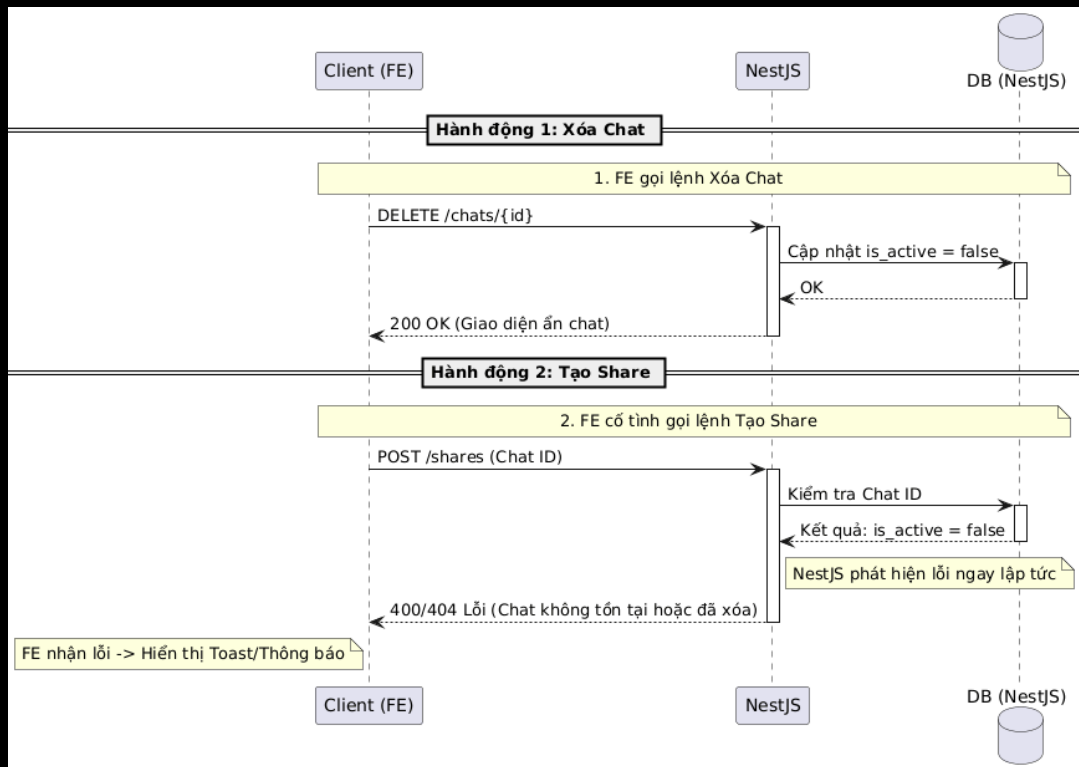
- **Khởi tạo trò chuyện:** Bắt đầu một luồng trò chuyện mới trong hệ thống.
- **Cấp phát token thoại:** Cấp phát token bảo mật phục vụ kết nối đàm thoại bằng giọng nói với máy chủ LiveKit.
- **Định tuyến tin nhắn:** Tiếp nhận tin nhắn từ người dùng và kết nối với Dịch vụ chatbot (chatbot service) thông qua giao thức gRPC để xử lý thông tin.
- **Xóa cuộc trò chuyện:** Hỗ trợ xóa mềm (soft delete) một cuộc trò chuyện khỏi hệ thống.
- **Quản lý thư mục:** Cho phép người dùng tạo các thư mục lưu trữ để phân loại và sắp xếp các cuộc trò chuyện.

- **Đánh dấu và Ghi chú:** Đánh dấu một cuộc trò chuyện quan trọng và ghi chú các nội dung tóm tắt đi kèm.
- **Chia sẻ trò chuyện:** Tạo ra một bản snapshot chia sẻ đoạn trò chuyện với người dùng khác.
- **Đường dẫn chia sẻ:** Cung cấp URL công khai chứa tham số `shareId` và token bảo mật đi kèm. Chủ sở hữu đoạn chat có thể xem danh sách các liên kết đã tạo và thu hồi quyền truy cập bất cứ lúc nào.

Chats			^
POST	/code-conversation-service/chats/start		🔒 ↓
POST	/code-conversation-service/chats/stream		🔒 ↓
DELETE	/code-conversation-service/chats/{chatId}		🔒 ↓
GET	/code-conversation-service/chats/{chat_id}/voice-token	Lấy token kết nối phiên hội thoại	🔒 ↓
Bookmarks			^
POST	/code-conversation-service/bookmarks		🔒 ↓
GET	/code-conversation-service/bookmarks		🔒 ↓
DELETE	/code-conversation-service/bookmarks/{folderId}		🔒 ↓
GET	/code-conversation-service/bookmarks/{folderId}		🔒 ↓
PUT	/code-conversation-service/bookmarks/{folderId}/item		🔒 ↓
DELETE	/code-conversation-service/bookmarks/{folderId}/item/{chatId}		🔒 ↓
Shared Chats			^
POST	/code-conversation-service/shared-chats		🔒 ↓
GET	/code-conversation-service/shared-chats		🔒 ↓
DELETE	/code-conversation-service/shared-chats/{shareId}		🔒 ↓
GET	/code-conversation-service/shared-chats/public/{shareId}/{token}		↓

Hình 0.13: Tài liệu Swagger API Dịch vụ trò chuyện

Sơ đồ trình tự kiểm duyệt nghiệp vụ khi xóa cuộc trò chuyện và chặn đứng việc chia sẻ cuộc trò chuyện đã bị xóa:

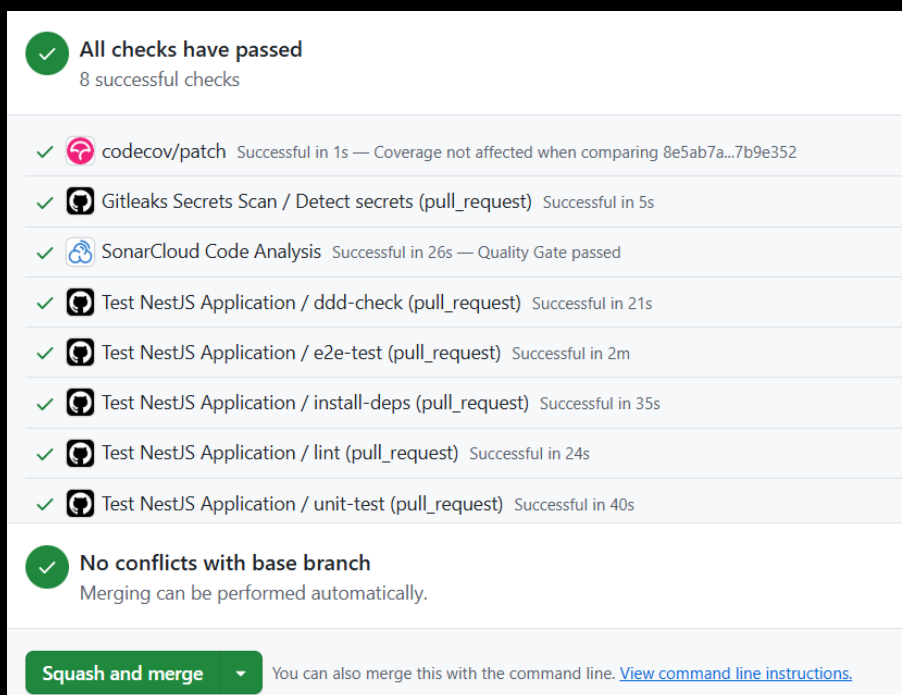


Hình 0.14: Sơ đồ trình tự xóa cuộc trò chuyện và kiểm duyệt chia sẻ

Chi tiết luồng xử lý và kiểm duyệt (Validation Logic):

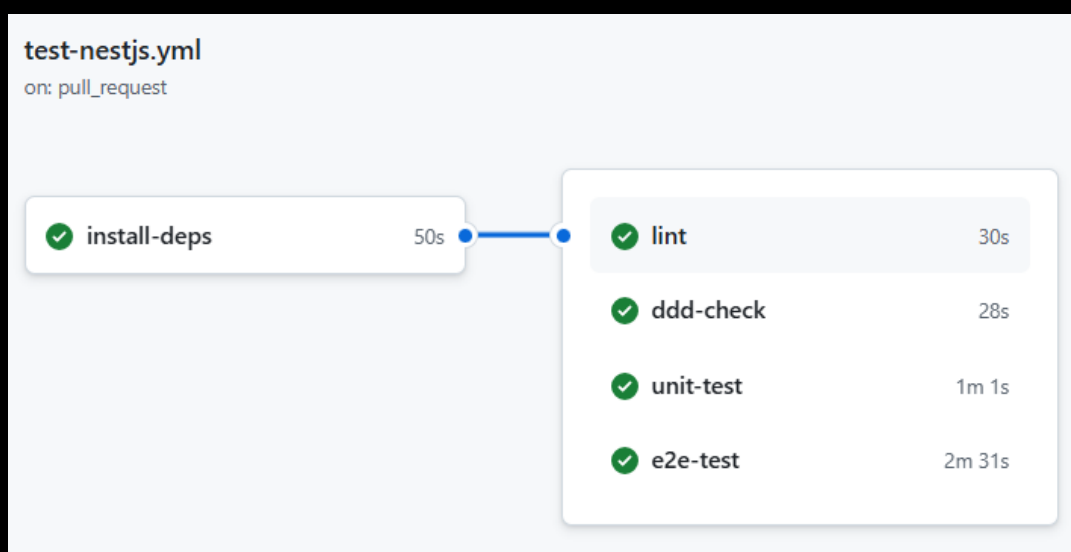
- Xóa cuộc trò chuyện (Delete Chat):** Client gửi yêu cầu `DELETE /chats/{id}` đến Dịch vụ trò chuyện (NestJS). Hệ thống tiến hành cập nhật trường trạng thái của cuộc trò chuyện trong Database thành `isActive = false` (thực hiện cơ chế xóa mềm - soft delete) và phản hồi `200 OK`. Lúc này, cuộc trò chuyện sẽ lập tức được ẩn đi trên giao diện người dùng.
- Kiểm duyệt tạo liên kết chia sẻ (Validate Share Link):** Khi client gửi yêu cầu `POST /shares` (kèm theo `chat_id`) để tạo link chia sẻ công khai, trước khi lưu bản ghi chia sẻ, `GenerateLinkShareCommandHandler` sẽ truy vấn thông tin cuộc trò chuyện từ Database dựa trên `chat_id`. Nếu cuộc trò chuyện không tồn tại hoặc có trạng thái `isActive = false` (đã bị xóa mềm), hệ thống sẽ chặn đứng hành động ngay lập tức và ném ra lỗi `ChatNotFoundException` (trả về mã lỗi `400/404` cho client kèm theo thông báo lỗi chi tiết trên giao diện UI).

Thông tin kiểm tra tự động Pull Request của dịch vụ trò chuyện trên GitHub:



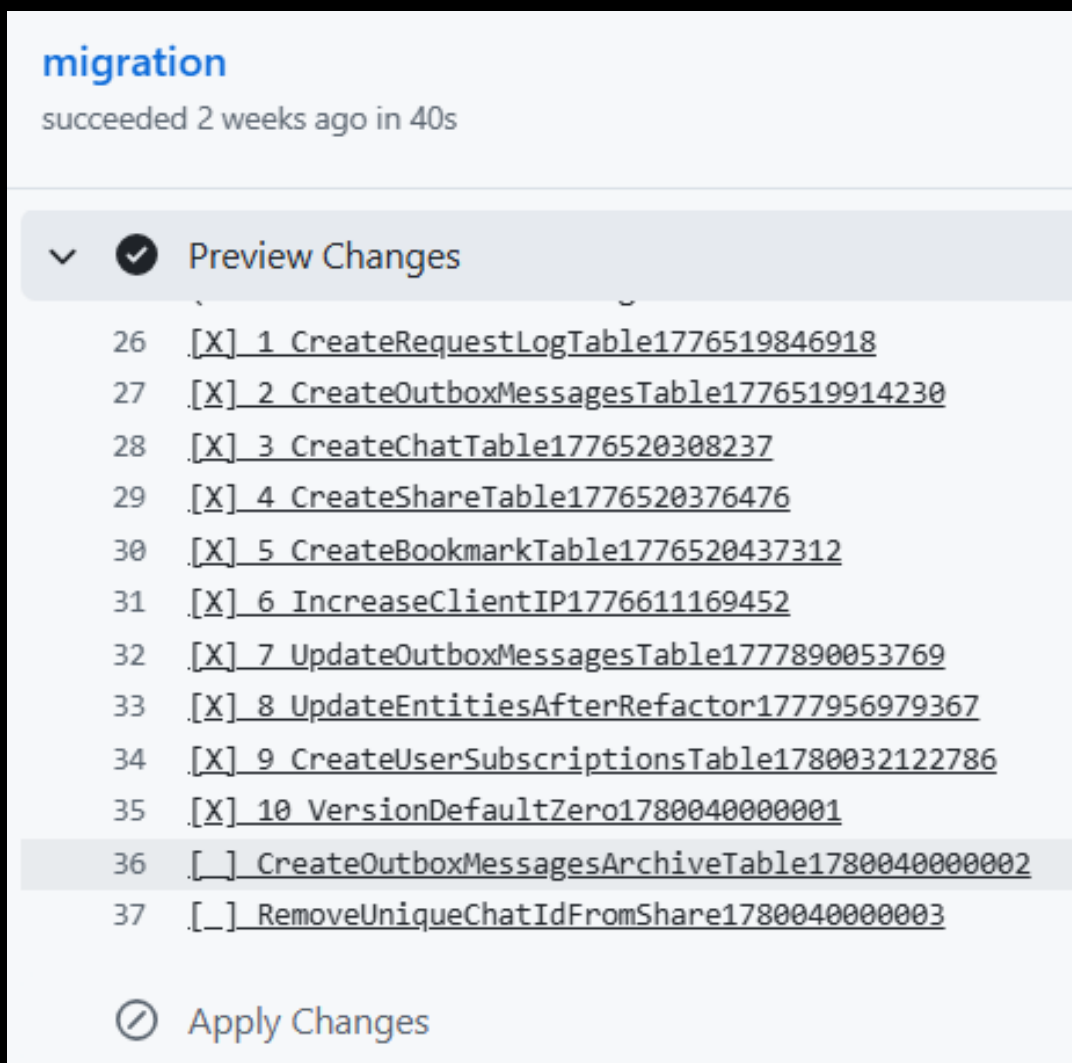
Hình 0.15: Kiểm tra Pull Request của conversation service trên GitHub

Kiểm tra cú pháp nguồn (linting) tự động trong CI/CD:



Hình 0.16: Kiểm tra code linting dịch vụ trò chuyện

Quản lý migrations CSDL của dịch vụ trò chuyện:



Hình 0.17: Quản lý database migrations dịch vụ trò chuyện

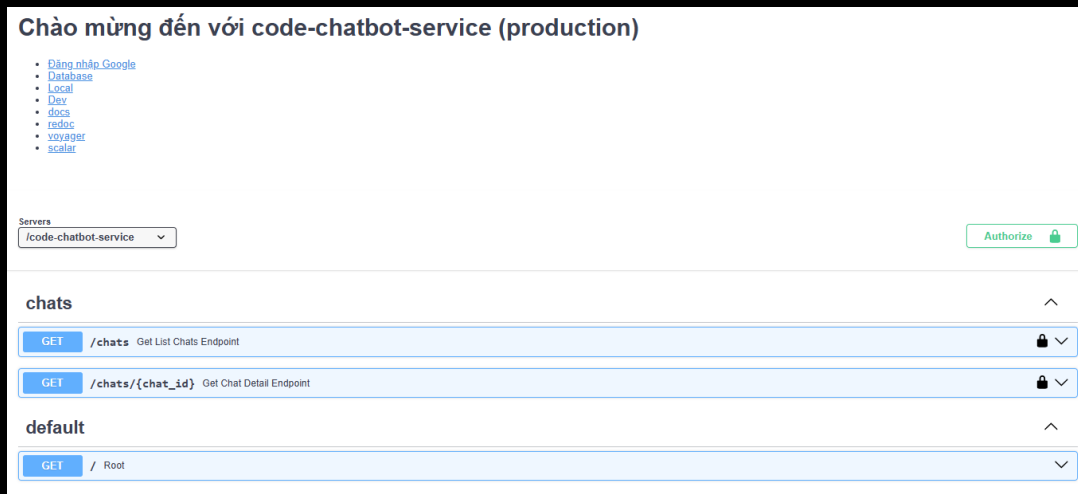
Cơ cấu các bảng trong CSDL của Dịch vụ trò chuyện:

Tables		DATA	STRUCTURE	Filters	Sort	Columns	+ Add record
neondb							
Schema							
public							
Search...							
bookmark_folder							
bookmark_item							
chat							
migrations							
outbox_messages							
outbox_messages_archive							
request_logs							
shared_chat							
user_subscriptions							

☐	id serial	timestamp bigint	name varchar	+
☐	1	1776519846918	CreateRequestLogTable1...	
☐	2	1776519914230	CreateOutboxMessagesTa...	
☐	3	1776520308237	CreateChatTable1776520...	
☐	4	1776520376476	CreateShareTable177652...	
☐	5	1776520437312	CreateBookmarkTable177...	
☐	6	1776611169452	IncreaseClientIP177661...	
☐	7	1777890053769	UpdateOutboxMessagesTa...	
☐	8	1777956979367	UpdateEntitiesAfterRef...	
☐	9	1780032122786	CreateUserSubscription...	
☐	10	1780040000001	VersionDefaultZero1780...	
☐	11	1780040000002	CreateOutboxMessagesAr...	
☐	12	1780040000003	RemoveUniqueChatIdFrom...	

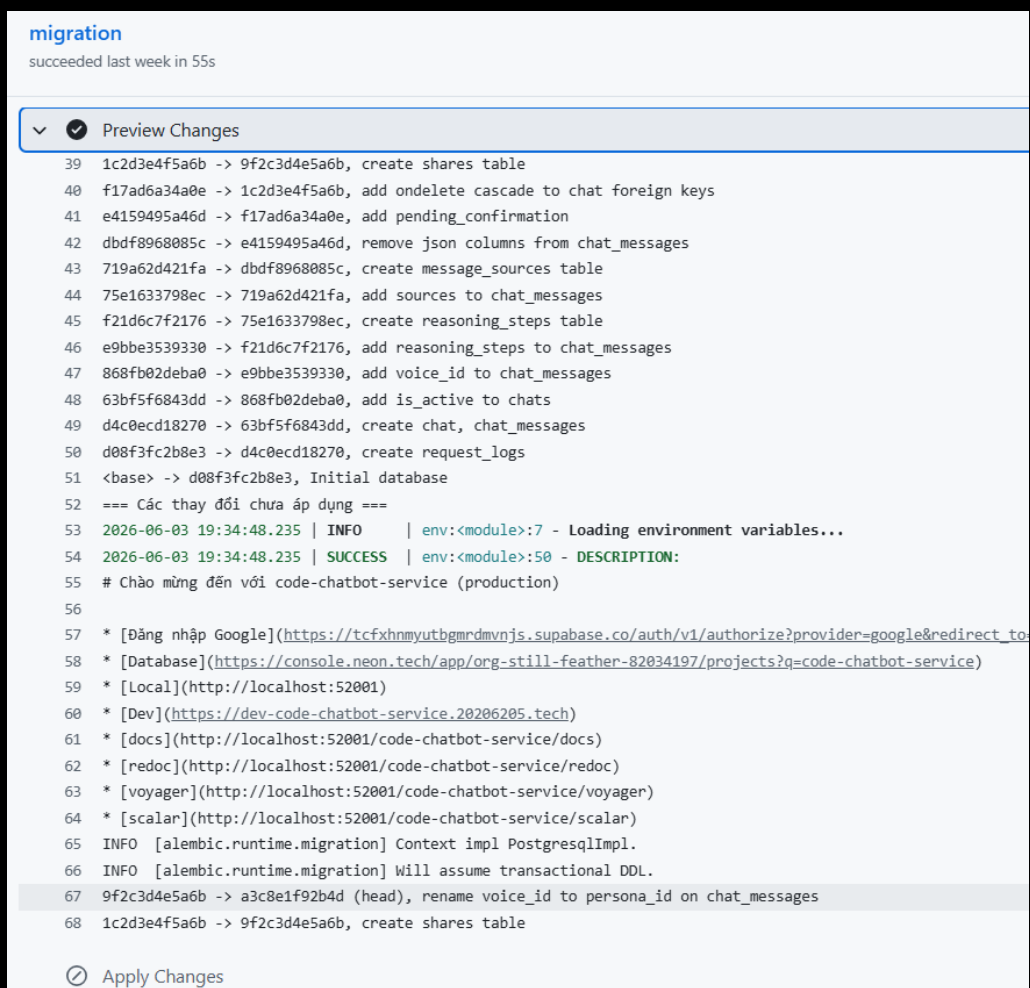
Hình 0.18: Cấu hình các bảng trong CSDL conversation

Dịch vụ chatbot (chatbot service) Mục đích: Sử dụng các mô hình AI để xử lý và phản hồi thông minh các câu hỏi của người dùng. Sau đó, lưu lại lịch sử trò chuyện và cung cấp các API truy xuất lịch sử, thông tin suy luận chi tiết (reasoning details) và nguồn tài liệu tham khảo (references) của từng câu trả lời. Dịch vụ chatbot lắng nghe RabbitMQ để thực hiện việc tạo và thu hồi các bản snapshot chia sẻ trò chuyện, đồng thời kết nối với LiveKit để vận hành đàm thoại giọng nói với người dùng.



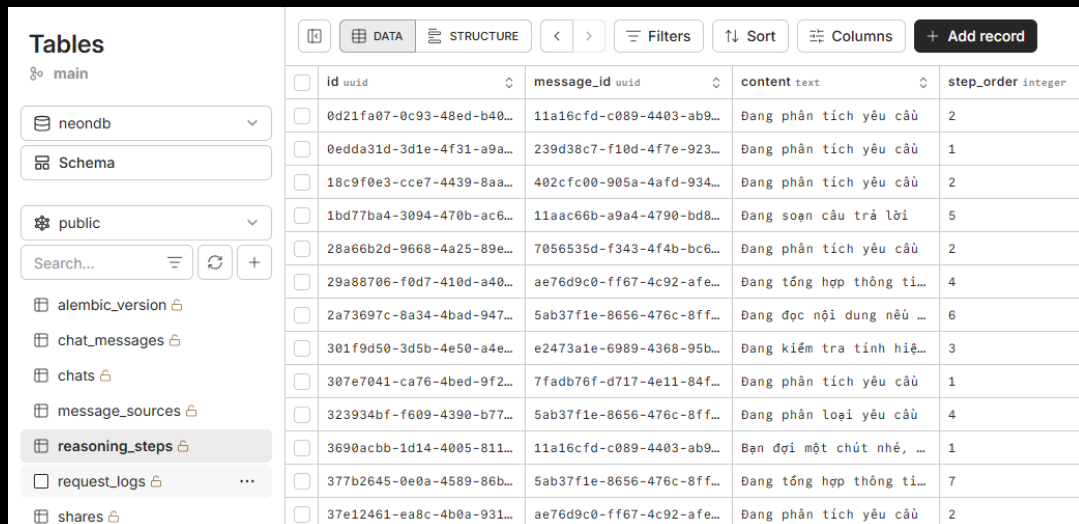
Hình 0.19: Tài liệu Swagger API Dịch vụ chatbot

Quản lý migrations CSDL của dịch vụ chatbot:



Hình 0.20: Quản lý database migrations dịch vụ chatbot

Cơ cấu các bảng trong CSDL của Dịch vụ chatbot:



id	message_id	content	step_order
0d21fa07-0c93-48ed-b40...	11a16cfd-c089-4403-ab9...	Đang phân tích yêu cầu	2
0edda31d-3d1e-4f31-a9a...	239d38c7-f10d-4f7e-923...	Đang phân tích yêu cầu	1
18c9f0e3-cc7-4439-8aa...	402cfc00-905a-4afd-934...	Đang phân tích yêu cầu	2
1bd77ba4-3094-470b-ac6...	11aac66b-a9a4-4790-bd8...	Đang soạn câu trả lời	5
28a66b2d-9668-4a25-89e...	7056535d-f343-4f4b-bc6...	Đang phân tích yêu cầu	2
29a88706-f0d7-410d-a40...	ae76d9c0-ff67-4c92-afe...	Đang tổng hợp thông ti...	4
2a73697c-8a34-4bad-947...	5ab37f1e-8656-476c-8ff...	Đang đọc nội dung nếu ...	6
301f9d50-3d5b-4e50-a4e...	e2473a1e-6989-4368-95b...	Đang kiểm tra tính hiệ...	3
307e7041-ca76-4bed-9f2...	7fad76f-d717-4e11-84f...	Đang phân tích yêu cầu	1
323934bf-f609-4390-b77...	5ab37f1e-8656-476c-8ff...	Đang phân loại yêu cầu	4
3690acbb-1d14-4005-811...	11a16cfd-c089-4403-ab9...	Bạn đợi một chút nhé, ...	1
377b2645-0e0a-4589-86b...	5ab37f1e-8656-476c-8ff...	Đang tổng hợp thông ti...	7
37e12461-ea8c-4b0a-931...	ae76d9c0-ff67-4c92-afe...	Đang phân tích yêu cầu	2

Hình 0.21: Cấu hình các bảng trong CSDL chatbot

Hệ thống trao đổi thông điệp qua RabbitMQ (RabbitMQ Messaging) Hệ thống sử dụng RabbitMQ làm Message Broker để thực hiện các giao tiếp bất đồng bộ, đáng tin cậy giữa **Dịch vụ trò chuyện (NestJS)** và **Dịch vụ chatbot (Python)**. Các tên hàng đợi được tiền tố hóa tự động theo môi trường (dev_, test_, prod_) dựa trên cấu hình hệ thống.

Danh sách các Hàng đợi trong hệ thống

- [prefix]_chat_deletions: Nhận yêu cầu xóa mềm (soft delete) lịch sử chat từ phía người dùng (Durable: true).
- [prefix]_share_generated: Yêu cầu tạo trang chia sẻ trò chuyện công khai (đóng gói JSONL và tải lên Cloudflare R2).
- [prefix]_share_generated_retry: Hàng đợi trung gian để xử lý lại (retry) khi gặp lỗi lệch thứ tự sự kiện (out of order). Cấu hình độ trễ cố định `x-message-ttl = 20s` trước khi gửi trả lại queue chính.
- [prefix]_share_generated_dlq: Dead Letter Queue nhận thông điệp tạo share thất bại hoàn toàn sau `MAX_RETRY_COUNT = 5` lần thử lại.
- [prefix]_share_upload_completed: Báo cáo kết quả đóng gói và upload file chia sẻ từ Python về lại NestJS.
- [prefix]_share_upload_completed_dlq: Dead Letter Queue nhận thông điệp báo cáo kết quả upload bị xử lý thất bại ở NestJS.
- [prefix]_share_revoked: Nhận yêu cầu thu hồi link chia sẻ, tiến hành xóa file tương ứng trên Cloudflare R2 và cập nhật DB (Durable: true).

Sự kiện Xóa trò chuyện

- Queue: [prefix]_chat_deletions
- Publisher: code-conversation-service (NestJS)
- Consumer: code-chatbot-service (Python)
- Payload Schema (JSON):

```
{
  "chat_id": "string", // Mã ID cuộc trò chuyện cần xóa
  "user_id": "string", // Mã ID người dùng sở hữu cuộc trò chuyện
  "version": 0 // Phiên bản sự kiện để kiểm tra thứ tự
}
```

Sự kiện Yêu cầu chia sẻ (Share Generated Event)

- **Queue:** [prefix]_share_generated
- **Publisher:** code-conversation-service (NestJS)
- **Consumer:** code-chatbot-service (Python)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID duy nhất của liên kết chia sẻ
  "chat_id": "string", // ID cuộc trò chuyện được chia sẻ
  "user_id": "string", // ID người dùng tạo chia sẻ
  "version": 1, // Phiên bản sự kiện (khởi tạo thường là 1)
  "last_message_id": "string" // ID tin nhắn cuối cùng
}
```

- **Cơ chế xử lý lệch thứ tự (Out-of-Order):** Consumer Python so sánh phiên bản sự kiện nhận được với phiên bản hiện tại trong DB. Nếu `event_version == current_version + 1`, tiến hành đóng gói lịch sử chat, upload R2 và gửi kết quả về NestJS. Nếu `event_version <= current_version`, bỏ qua vì đã xử lý. Nếu `event_version > current_version + 1`, sự kiện đến sai thứ tự, ném ra ngoại lệ `OutOfOrderException` để đưa tin nhắn vào `retry` tạm chờ 20 giây trước khi thử lại. Sau 5 lần thất bại liên tục sẽ bị chuyển vào DLQ và gửi cảnh báo về hệ thống.

Sự kiện Thu hồi liên kết (Share Revoked Event)

- **Queue:** [prefix]_share_revoked
- **Publisher:** code-conversation-service (NestJS)
- **Consumer:** code-chatbot-service (Python)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID liên kết chia sẻ cần thu hồi
  "file_url": "string", // URL tệp tin JSONL cần xóa trên Cloudflare R2
  "version": 2 // Phiên bản sự kiện (thường là phiên bản kế tiếp)
}
```

- **Cơ chế xử lý:** Xóa tệp tin trên Cloudflare R2, đánh dấu `is_revoked = True` trong DB. Hỗ trợ cơ chế tự động gửi lại hàng đợi chính (requeue) nếu phát hiện sự kiện đến lệch thứ tự (revoke đến trước khi generate kịp ghi DB).

Sự kiện Hoàn tất Upload (Share Upload Completed Event)

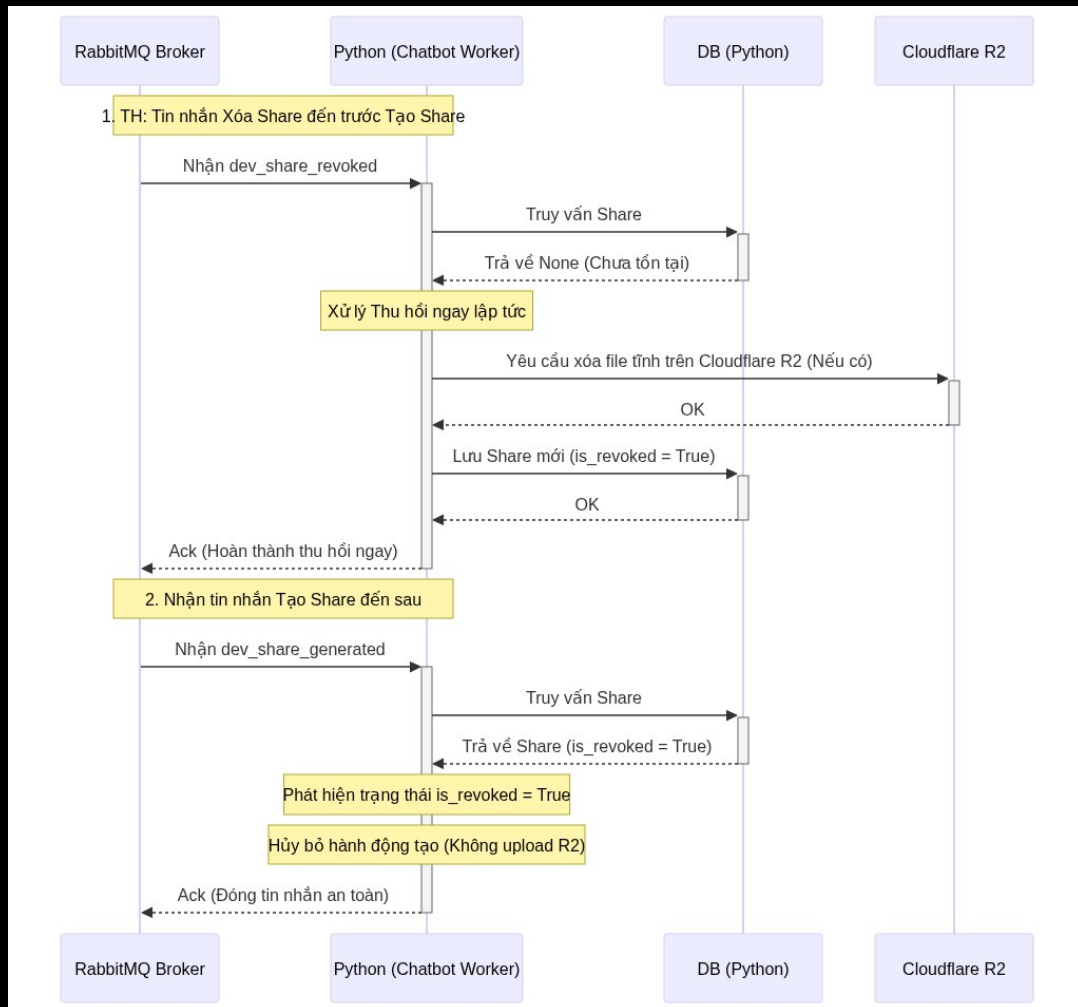
- **Queue:** [prefix]_share_upload_completed
- **Publisher:** code-chatbot-service (Python)
- **Consumer:** code-conversation-service (NestJS)
- **Payload Schema (JSON):**

```
{
  "share_id": "string", // ID liên kết chia sẻ tương ứng
  "file_url": "string", // URL tệp tin JSONL
  "is_success": true // Trạng thái thành công (true / false)
}
```

- **Cơ chế xử lý:** NestJS nhận thông điệp sẽ tiến hành cập nhật trạng thái của bản ghi chia sẻ thành công hoặc thất bại trong DB (sử dụng Manual Ack để đảm bảo không bị mất dữ liệu).

Xử lý lệch thứ tự sự kiện Tạo Share và Thu hồi/Xóa Share (Out-of-Order Handling)

- **Tình huống:** Người dùng vừa tạo Share và lập tức thực hiện xóa/thu hồi Share đó. Do tính chất bất đồng bộ của Message Broker, sự kiện Thu hồi Share (`share_revoked`) có thể đến Python worker **trước** sự kiện Tạo Share (`share_generated`).
- **Cơ chế kiểm tra trạng thái Lũy đẳng (State-based Idempotency):** Vì việc thu hồi liên kết thường liên quan đến vấn đề rò rỉ thông tin nhạy cảm của người dùng, nghiệp vụ yêu cầu lệnh thu hồi phải được thực hiện **ngay lập tức và ưu tiên cao nhất**. Do đó, hệ thống không áp dụng cơ chế tuần tự kiểm tra version phức tạp mà sử dụng cờ trạng thái `is_revoked` để triệt tiêu sự kiện:



Hình 0.22: Sơ đồ xử lý lệch thứ tự tạo và thu hồi share

1. **Khi sự kiện Thu hồi Share đến trước:** Python kiểm tra trạng thái trong DB (chưa có bản ghi nào), tiến hành gọi xóa tệp tin trên Cloudflare R2 (nếu đã được tạo trước đó), lưu bản ghi Share vào DB với trạng thái `is_revoked = True` và phản hồi Ack để hoàn thành tin nhắn.
2. **Khi sự kiện Tạo Share đến sau:** Python kiểm tra DB và phát hiện bản ghi đã tồn tại với trạng thái `is_revoked = True`. Khi đó, áp dụng **Cơ chế Triệt tiêu sự kiện (Event Cancellation)**: lập tức bỏ qua (drop) hoàn toàn hành động tạo và upload tệp lên R2 để bảo vệ tuyệt đối dữ liệu nhạy cảm của người dùng, sau đó gửi Ack để đóng tin nhắn một cách an toàn.

Thiết lập hàng đợi ưu tiên (Priority Queue) trên RabbitMQ

Để đảm bảo các thông điệp thu hồi khẩn cấp được đẩy lên xử lý trước trong trường hợp hàng đợi bị nghẽn (backlog), hệ thống hỗ trợ tích hợp **Priority Queue**:

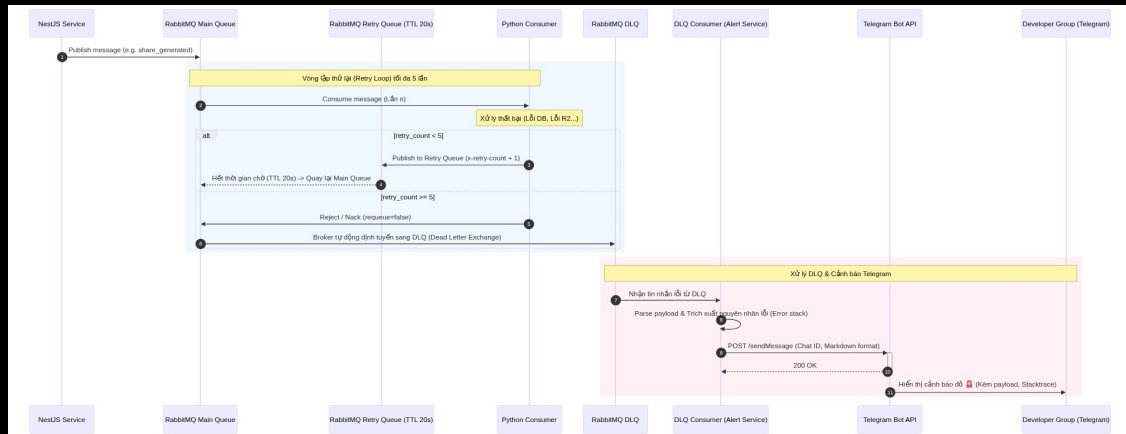
1. **Khai báo hàng đợi hỗ trợ ưu tiên:** Khi khởi tạo queue trên RabbitMQ, gán đối số cấu hình mức ưu tiên cao nhất (khuyến nghị tối đa là 10 để tránh tốn tài nguyên RAM/CPU):

```
channel.queue_declare(
    queue='[prefix]_share_events_queue',
    arguments={'x-max-priority': 10}
)
```

- Gửi sự kiện kèm mức độ ưu tiên (Producer):** Sự kiện Thu hồi Share (`share_revoked`) được gửi với độ ưu tiên khẩn cấp (`priority = 10`); Sự kiện Tạo Share (`share_generated`) được gửi với độ ưu tiên bình thường (`priority = 1`).
- Cấu hình QOS cho Consumer:** Thiết lập thuộc tính `prefetch_count = 1` trên consumer để bắt buộc RabbitMQ chỉ gửi đúng 1 thông điệp tại một thời điểm cho worker. Điều này giúp hàng đợi có cơ hội sắp xếp và đẩy các tin nhắn có độ ưu tiên cao (`priority = 10`) lên xử lý trước.

Luồng xử lý hàng đợi thư chết (DLQ) kết hợp Cảnh báo Telegram

- Mục đích:** Khi một tin nhắn gặp lỗi xử lý lặp đi lặp lại nhiều lần (lỗi logic, lỗi kết nối CSDL kéo dài, lỗi phân tích cú pháp dữ liệu), hệ thống cần có lập tin nhắn đó để tránh làm nghẽn hàng đợi chính, đồng thời phát cảnh báo tức thời cho đội ngũ vận hành (Developers/DevOps).
- Quy trình hoạt động:** Sự kết hợp giữa cơ chế Dead Letter Queue (DLQ) của RabbitMQ và Telegram Bot API được thực hiện như sau:



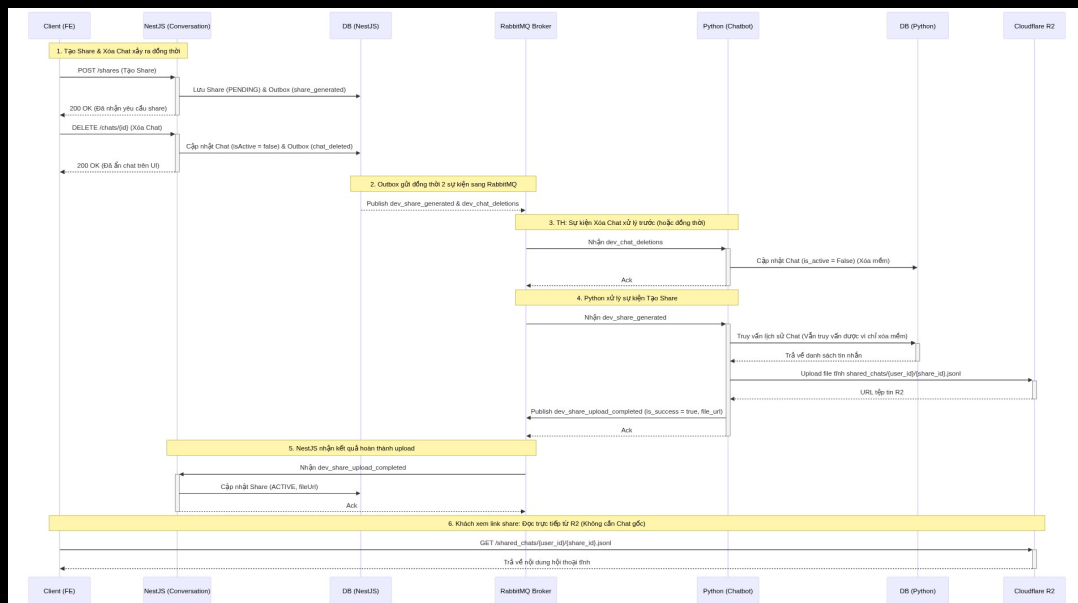
Hình 0.23: Sơ đồ luồng xử lý DLQ và thông báo Telegram

- Ghi nhận và Thử lại (Retry Loop):** Khi consumer (Python/NestJS) xử lý tin nhắn thất bại, tin nhắn sẽ được gửi sang hàng đợi Retry với độ trễ (TTL) cố định là 20 giây, đồng thời số lần thử lại (`x-retry-count`) được lưu và tăng dần trong header của tin nhắn.
- Chuyển sang DLQ:** Sau khi thử lại thất bại quá số lần quy định (`MAX_RETRY_COUNT = 5`), consumer sẽ từ chối tin nhắn không điều kiện (`message.reject(requeue=False)`). RabbitMQ sẽ tự động chuyển hướng tin nhắn này sang hàng đợi thư chết tương ứng (`_dlq`) dựa trên cấu hình `x-dead-letter-routing-key` lúc khởi tạo queue.
- Tiêu thụ DLQ và Cảnh báo:** Một consumer chuyên biệt (`process_share_generated_dlq`) sẽ lắng nghe hàng đợi DLQ. Khi có tin nhắn đi vào DLQ, consumer này sẽ trích xuất thông tin lỗi, payload của sự kiện, thời gian xảy ra sự cố và gửi yêu cầu HTTP POST tới Telegram Bot API để thông báo tức thời cho đội ngũ vận hành.

Xử lý concurrency khi Tạo Share và Xóa Chat đồng thời (Concurrency & Snapshot Architecture) Trong thực tế vận hành, người dùng có thể thực hiện thao tác tạo liên kết chia sẻ (Share) và ngay lập tức xóa cuộc trò chuyện (Delete Chat). Lúc này, cơ chế Outbox sẽ đẩy đồng thời hai sự kiện `share_generated` và `chat_deleted` vào RabbitMQ. Để giải quyết bài toán Race Condition (xung đột tài nguyên) khi thứ tự xử lý trên RabbitMQ không cố định, hệ thống áp dụng mẫu kiến trúc **Snapshot/Static Generation**:

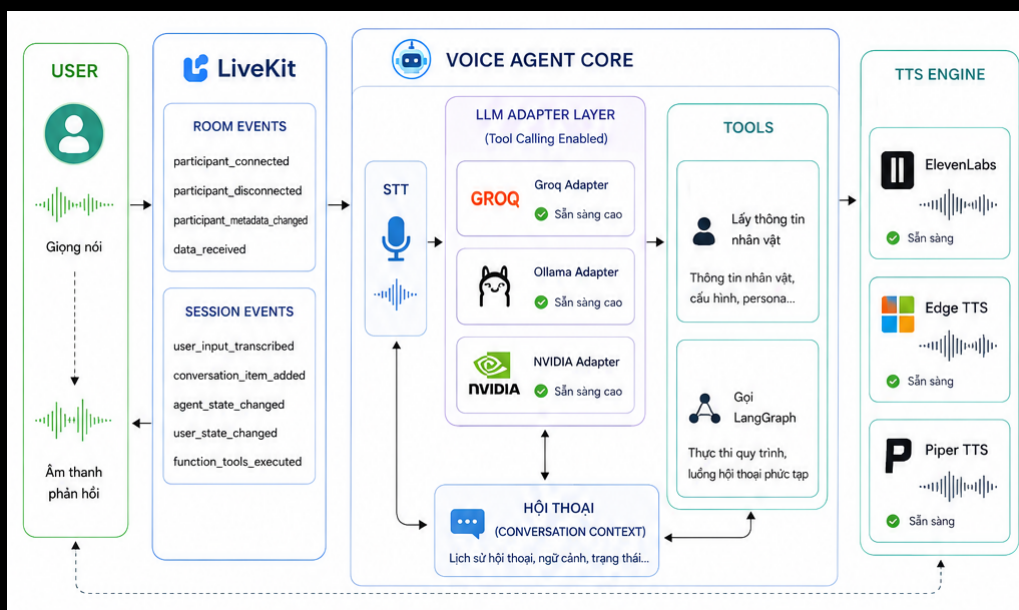
Cơ chế hoạt động chi tiết:

- **Không phụ thuộc vào trạng thái hoạt động của Chat:** Khi NestJS hoặc Python nhận sự kiện xóa chat, họ chỉ thực hiện **xóa mềm (soft delete)** bằng cách cập nhật trường `isActive = false/is_active = False` trong CSDL. Bản ghi và dữ liệu tin nhắn vẫn tồn tại vật lý trong database.
- **Tạo Snapshot tĩnh (Static Generation):** Dù sự kiện `chat_deleted` có được xử lý trước sự kiện `share_generated` trên Python worker, Python vẫn có thể truy vấn đầy đủ nội dung lịch sử chat đã xóa mềm từ database để đóng gói thành tệp JSONL tĩnh và tải lên Cloudflare R2.
- **Tính độc lập của liên kết chia sẻ:** Sau khi upload thành công lên Cloudflare R2, liên kết chia sẻ trở trực tiếp đến tệp tĩnh này. Khi người nhận xem liên kết chia sẻ, Frontend sẽ fetch trực tiếp tệp tĩnh từ R2 (qua CDN) mà không cần truy vấn vào database gốc (Neon) hay đi qua NestJS. Do đó, nghiệp vụ chấp nhận trạng thái "Có liên kết Share hoạt động nhưng Chat gốc đã bị xóa".



Hình 0.24: Sơ đồ xử lý đồng thời tạo share và xóa chat

Chi tiết chức năng đàm thoại giọng nói (Voice Agent) Hệ thống thiết lập các bộ chuyển đổi (Adapters) cho phép kết nối linh hoạt với nhiều nhà cung cấp mô hình ngôn ngữ (Groq, Ollama và NVIDIA) có hỗ trợ gọi công cụ (Tool Calling) để đảm bảo tính sẵn sàng cao của hệ thống. AI Agent tích hợp sẵn công cụ lấy thông tin nhân vật (Get Persona) và công cụ gọi luồng xử lý LangGraph để phản hồi câu hỏi. Hệ thống hỗ trợ tự động chuyển đổi công cụ TTS giữa ElevenLabs, Edge TTS và Piper TTS để đáp ứng linh hoạt theo cấu hình nhân vật của người dùng. Sơ đồ tổng quan luồng tích hợp LiveKit đàm thoại giọng nói:



Hình 0.25: Sơ đồ tích hợp LiveKit đàm thoại giọng nói

Các sự kiện của LiveKit được hệ thống Voice Agent tiếp nhận và xử lý:

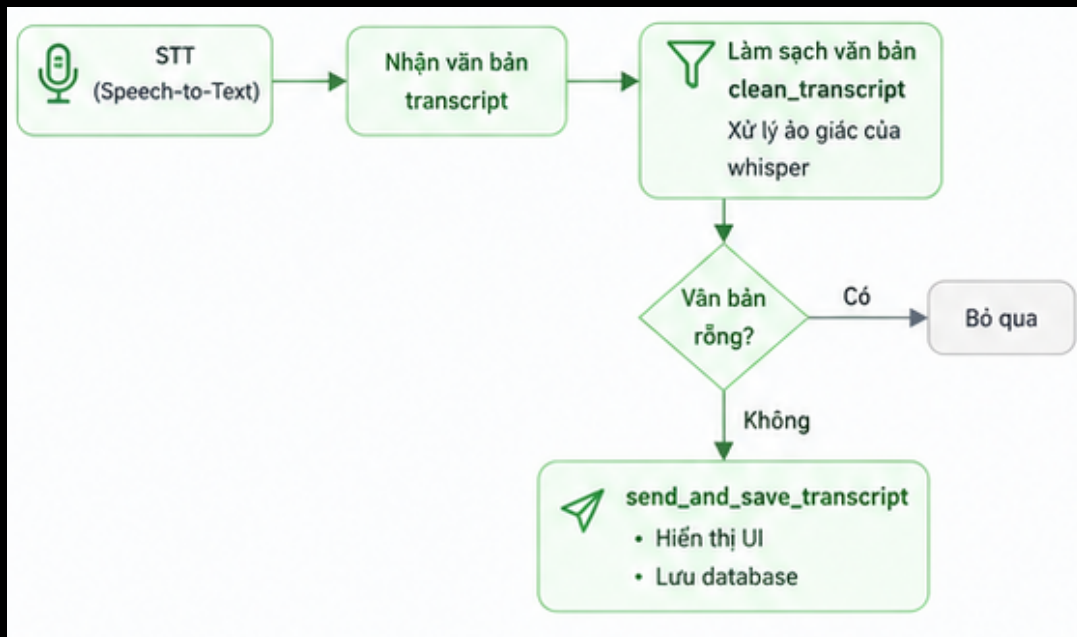
Sự kiện phòng (Room Events) Quản lý vòng đời kết nối và sự hiện diện của người dùng trong phòng đàm thoại:

- **participant_connected**: Kích hoạt khi người dùng tham gia vào phòng đàm thoại thành công. Hệ thống tự động gọi hàm chủ động phát giọng nói chào hỏi người dùng (Lần đầu kết nối: `WELCOME_MESSAGE` = "Xin chào, tôi là trợ lý pháp luật. Bạn có cần tôi giúp đỡ gì không?", Các lần kết nối lại sau đó: `REJOIN_MESSAGE` = "Chào mừng bạn đã quay trở lại. Bạn có cần tôi giúp đỡ gì không?").
- **participant_disconnected**: Kích hoạt khi người dùng rời khỏi phòng hoặc bị ngắt kết nối đột ngột.
- **participant_metadata_changed**: Lắng nghe sự thay đổi siêu dữ liệu (metadata) từ phía người dùng để cập nhật cấu hình giọng đọc, nhân vật mới một cách tức thời.
- **data_received**: Nhận dữ liệu truyền qua kênh data channel (thường là tin nhắn văn bản định dạng JSON). Cho phép nhà phát triển (developer) gửi tin nhắn văn bản thay cho âm thanh nói để debug luồng xử lý của AI một cách thuận tiện.

Sự kiện phiên thoại (Session Events) Quản lý các sự kiện trong luồng đàm thoại giữa người dùng và AI Agent:

- **user_input_transcribed**: Nhận văn bản sau khi hệ thống Speech-to-Text (STT) chuyển đổi thành công giọng nói của người dùng. Áp dụng bộ lọc `clean_transcript` để loại bỏ các đoạn văn bản ảo giác của Whisper, bỏ qua nếu chuỗi rỗng và gọi `send_and_save_transcript` để hiển thị nội dung văn bản tương ứng lên giao diện UI và lưu vào CSDL.
- **conversation_item_added**: Kích hoạt mỗi khi có một tin nhắn mới (từ người dùng hoặc AI Assistant) được thêm vào ngữ cảnh trò chuyện. Đối với phản hồi từ AI Assistant, hệ thống tiến hành trích xuất nội dung tin nhắn, kiểm tra và lọc bỏ các bước suy luận trung gian của LangGraph. Khi có câu trả lời cuối cùng, hệ thống gọi `send_and_save_transcript` để hiển thị trên UI và lưu CSDL.
- **agent_state_changed** và **user_state_changed**: Theo dõi sự thay đổi trạng thái hoạt động (như Speaking, Listening, Thinking) của cả AI Agent và người dùng để đồng bộ hiển thị trạng thái động trên UI.
- **function_tools_executed**: Kích hoạt khi AI Agent quyết định gọi một công cụ bên ngoài (Function/Tool Calling) thành công.

Bộ lọc làm sạch văn bản `clean_transcript` xử lý loại bỏ ảo giác trong văn bản dịch từ giọng nói:



Hình 0.26: Bộ lọc làm sạch văn bản `clean_transcript`

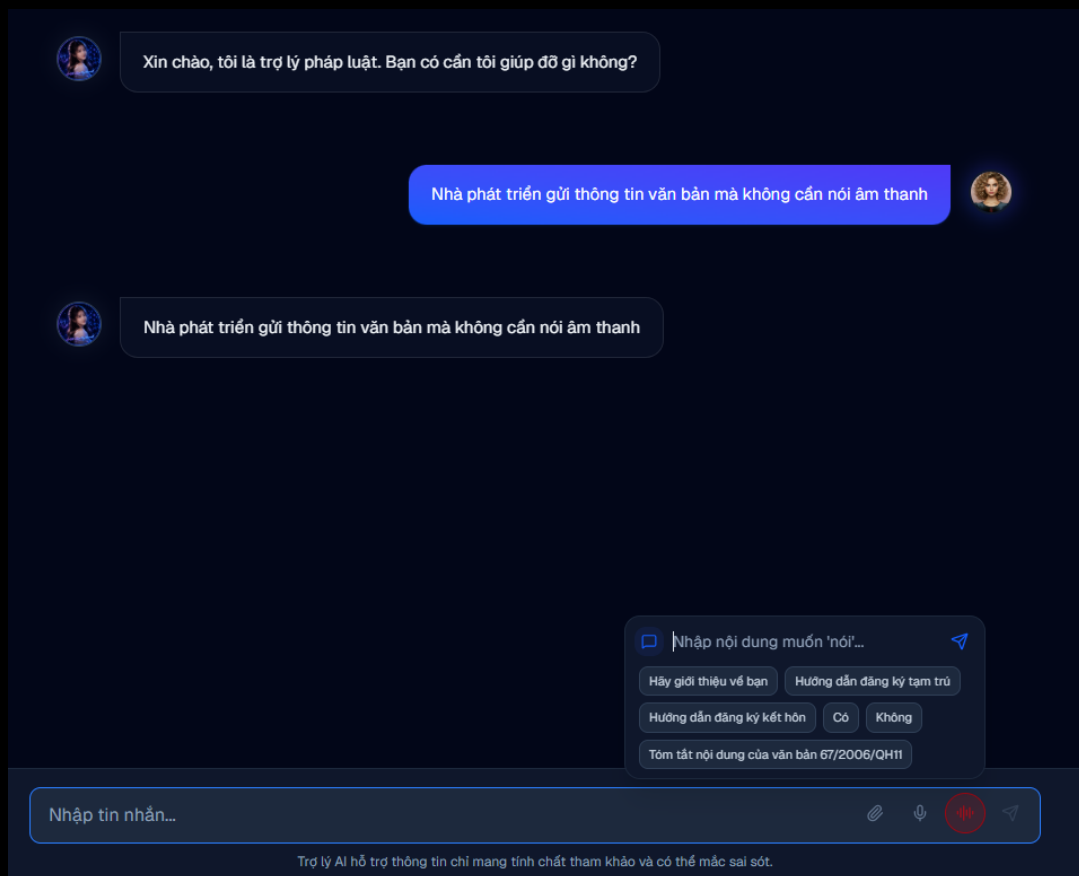
Quản lý các file âm thanh ghi âm trò chuyện lưu trữ trên Cloudflare R2:

[dev-share](#) / [shared_chats](#) / c36ce92d-704d-452a-883e-67a984645e4e /

☐ Objects	Type	Storage Class
☐ 01c2cadb-6450-4534-b173-f21190b8b7a0.jsonl	application/x-ndjson; cha...	Standard
☐ 01ee2872-36bb-4491-8c4f-283f09e5eee5.jsonl	application/x-ndjson; cha...	Standard
☐ 35945422-ccbf-4b91-8a0a-00fd1d5d5766.jsonl	application/x-ndjson; cha...	Standard
☐ 451ad427-7eb1-4d99-95be-578b0e57fc12.jsonl	application/x-ndjson; cha...	Standard
☐ 5811c0d9-d658-4e8b-9cdd-bf394ec3f10b.jsonl	application/x-ndjson; cha...	Standard
☐ 5c04c7a3-0ba8-44c6-83d6-9f7dafb1e91e.jsonl	application/x-ndjson; cha...	Standard

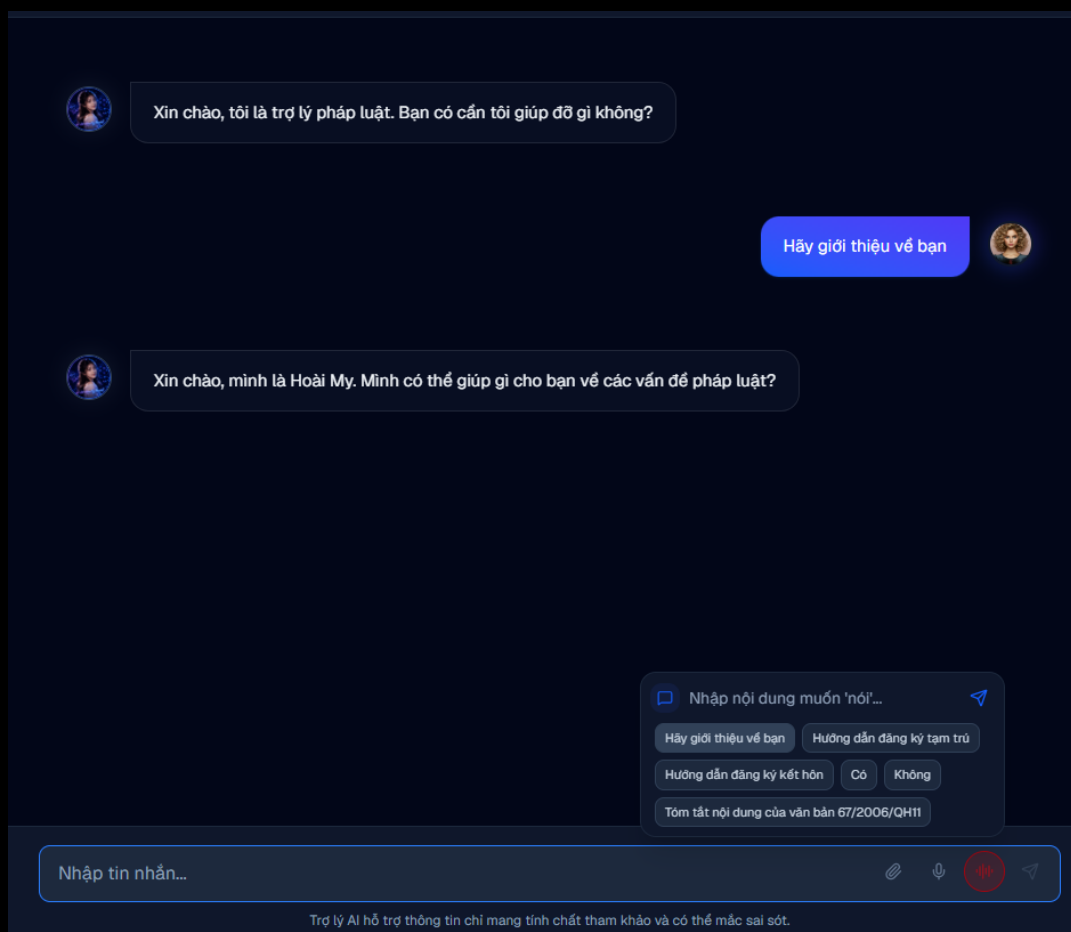
Hình 0.27: Lưu trữ các tệp tin âm thanh chat trên Cloudflare R2

Nhà phát triển gửi thông tin văn bản qua data channel mà không cần nói âm thanh:



Hình 0.28: Developer gửi tin nhắn văn bản thay giọng nói để debug

AI Agent sử dụng công cụ lấy thông tin nhân vật từ hệ thống:



Hình 0.29: Giao diện lấy thông tin nhân vật từ API Gateway

Thông tin log terminal khi AI thực thi công cụ lấy thông tin nhân vật thành công:

```
2026-06-15 01:24:21.512 | DEBUG | app.voice.events.session.on_function_tools_executed:
on_function_tools_executed:5 - AI vừa sử dụng công cụ: type='function_tools_executed' fun
ction_calls=[FunctionCall(id='item_e82d4166f61c/fnc_0', type='function_call', call_id='fc
_3c032004-6093-48e4-913f-62646d31a182', arguments={}, name='get_persona_info', created
_at=1781461461.1885478, extra={}, group_id=None)] function_call_outputs=[FunctionCallOutpu
t(id='item_d388c3411560', type='function_call_output', name='get_persona_info', call_id='
fc_3c032004-6093-48e4-913f-62646d31a182', output='Xin chào, mình là Hoài My. Mình có thể
giúp gì cho bạn về các vấn đề pháp luật?', is_error=False, created_at=1781461461.5105371)
] created_at=1781461461.5115511
2026-06-15 01:24:23.007 - WARNING livekit.agents - no request_id provided for TTS livekit
.plugins.openai.tts.TTS
2026-06-15 01:24:23.327 - WARNING livekit.agents - no request_id provided for TTS livekit
.plugins.openai.tts.TTS
2026-06-15 01:24:28.977 - DEBUG livekit.agents - conversation_item_added {"role": "assist
ant", "text": "Xin chào, mình là Hoài My. Mình có thể giúp gì cho bạn về các vấn đề pháp
luật?"}
2026-06-15 01:24:28.977 | WARNING | app.voice.events.session.on_conversation_item_added:
on_conversation_item_added:45 - 🐱 Agent (Final): Xin chào, mình là Hoài My. Mình có thể
giúp gì cho bạn về các vấn đề pháp luật?
```

Hình 0.30: Terminal log của việc gọi công cụ lấy thông tin nhân vật

Cơ chế Lập lịch Dọn dẹp dữ liệu (Data Cleanup Scheduling) Vì tính năng xóa trò chuyện và thu hồi liên kết chia sẻ trong hệ thống sử dụng cơ chế **xóa mềm (soft delete)** để đảm bảo tính an toàn dữ liệu và tối ưu hiệu năng của các giao dịch tức thời, hệ thống triển khai các tác vụ lập lịch định kỳ (cron jobs / scheduler) trên cả **NestJS** và **Python** để dọn dẹp triệt để các dữ liệu đã bị xóa sau thời gian lưu trữ quy định.

Phía Dịch vụ trò chuyện (NestJS - code-conversation-service) NestJS sử dụng module `@nestjs/schedule` để quản lý các cron jobs dọn dẹp các bản ghi không còn hoạt động quá **30 ngày**:

- **ChatCleanupCron**: Tìm và xóa vĩnh viễn các bản ghi Chat có `isActive = false` và thời gian cập nhật cách hiện tại từ 30 ngày trở lên. Chu kỳ chạy: Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **ShareCleanupCron**: Tìm và xóa vĩnh viễn các bản ghi liên kết Share có `isActive = false` (hoặc đã bị thu hồi) quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày vào lúc nửa đêm (môi trường production) hoặc mỗi 10 phút (môi trường development).

Phía Dịch vụ chatbot (Python - code-chatbot-service) Python sử dụng thư viện `APScheduler` (`AsyncIOScheduler`) chạy nền tích hợp trong vòng đời (lifespan) của ứng dụng để tự động dọn dẹp CSDL lưu trữ lịch sử:

- **run_chat_cleanup**: Xóa vĩnh viễn các bản ghi Chat trong CSDL chatbot có trạng thái `is_active = False` quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).
- **run_share_cleanup**: Xóa vĩnh viễn các bản ghi Share trong CSDL chatbot có trạng thái `is_revoked = True` quá hạn 30 ngày. Chu kỳ chạy: Mỗi ngày lúc 12:00 trưa (môi trường production) hoặc mỗi 10 phút (môi trường development).